DevOps-путеводитель по проекту «Цифровая очередь»

> Учебный разбор состояния репозитория на 2026-10-01. Это не инструкция по
>
> публикации секретов: пароли, API-токены, kubeconfig и IP-адреса намеренно
>
> заменены на <...> . Их нельзя хранить в Git.

1. Главное за две минуты

В проекте есть **две альтернативные среды запуска** и несколько слоёв вокруг

них:

```
Локально / на одном Docker-хосте
  .env → Docker Compose → containers → Nginx → пользователь
                └→ Prometheus / Loki / Grafana / Alertmanager

Kubernetes-путь (задуман как production)
  Git → Argo CD → Helm render → K3s → Pods + Services → Traefik Ingress
                                    └→ Prometheus / Loki / Jaeger / Grafana
```

Docker упаковывает приложение и зависимости в образы.

Docker Compose поднимает весь набор контейнеров на одной машине.

Ansible по SSH готовит машины и ставит K3s/Fail2ban.

K3s — облегчённая дистрибуция Kubernetes; она запускает контейнеры как

Pods и даёт встроенный ingress-controller Traefik.

Kubernetes описывается декларативными YAML: желаемое состояние, а не

последовательность команд.

Helm — шаблонизатор и пакетный менеджер Kubernetes. Из `templates/` и `values.yaml` он получает обычные Kubernetes YAML.

Argo CD следит за Git и приводит кластер к версии манифестов в Git (GitOps pull-модель).

Prometheus собирает числовые метрики, **Alertmanager** рассылает алерты, **Loki + Promtail** собирают логи, **Grafana** отображает всё, **Jaeger + OpenTelemetry** показывают путь одного запроса (trace).

2. Как проходит один HTTP-запрос

```
Browser → Nginx (Docker-вариант) или Traefik Ingress (K8s-вариант)
        ├→ /     → frontend Service/контейнер
        └→ /api  → backend Service/контейнер → PostgreSQL

backend: JSON log (trace_id) —→ stdout —→ Promtail —→ Loki —→ Grafana
backend: /metrics            ————————→ Prometheus ————————→ Grafana/Alertmanager
backend: span (OTLP/HTTP)    ————————→ Jaeger ———————————→ Jaeger/Grafana
```

`/health` отвечает, что HTTP-процесс жив. `/ready` дополнительно проверяет PostgreSQL: Kubernetes не включает pod в балансировку Service, пока readiness не проходит.

3. Сначала разберитесь, какой путь реально активен

Источник истины	Для чего	Как применяется
`docker-compose.yml`	Локальная/односерверная среда	`docker compose up`
`k8s/*.yaml`	Сырые K8s-манифесты, ручной путь	`kubectl apply -f …`
`helm/queue-chart/`	Шаблонный K8s-релиз	`helm upgrade --install …` или Argo CD
`k8s/argocd-app.yaml`	Указывает Argo CD использовать Helm chart	один раз применить в namespace `argocd`

Не следует применять одновременно `k8s/backend-deployment.yaml` и Helm chart: они создают одноимённые `Deployment` / `Service`, но с разными версиями образа и настройками. Для GitOps источником истины должен быть Helm chart, потому что именно на него ссылается Argo Application.

4. Контейнеры: Dockerfile, строка за строкой

`backend/Dockerfile`

Строки	Что означает
1	Первый этап multi-stage: Go-компилятор на Alpine.
2	Рабочая папка в build-контейнере.
4–5	Копируются модули и скачиваются зависимости; это отдельный слой cache.
6–7	Копируется исходный код; собирается статический (`CGO_ENABLED=0`) бинарник `api` . `-trimpath` , `-s` , `-w` уменьшают образ и не вшивают пути сборки.
9–10	Второй, runtime-этап. В итоговый образ не попадает Go toolchain.
11, 15	Создаётся и выбирается непривилегированный пользователь `app` — правильнее, чем запуск от root.
12–14	В образ копируются бинарник, SQL-миграции и файл политики очереди.
16	`ENTRYPOINT` : при старте контейнера всегда запускается `/app/api` ; аргумент Compose `notify-demo` становится аргументом бинарника.

`frontend/Dockerfile` и `frontend/Dockerfile.prod`

Файл, строки	Что происходит
`frontend/Dockerfile:1–4`	Dev-образ: Node, `WORKDIR` , lockfile и воспроизводимый `npm ci` .
`:5–8`	Копия `node_modules` сохраняется в `/opt` : Compose может восстановить named volume после чистого checkout.
`:9–11`	Копирует исходники, открывает 5173, запускает Vite dev server. Это **не production-образ**.
`frontend/Dockerfile.prod:1–6`	Build-stage собирает статический Vite `dist` .
`:8–12`	Runtime-stage Nginx получает только `dist` и собственный конфиг, слушает 80. Такой образ предназначен для Kubernetes.
`frontend/nginx.conf:1–9`	SPA fallback: неизвестный URL возвращает `index.html` , чтобы роутинг React работал после refresh.
`:11–14`	Небольшой endpoint `/healthz` для liveness/readiness probe.

Python-сервисы (`services/ml-service/Dockerfile` , `services/rag-agent/Dockerfile`)

устроены одинаково: базовый Python, зависимости, код, `uvicorn` на 8090/8095.

Они есть в Compose; Helm chart сейчас их **не создаёт**.

5. Docker Compose: `docker-compose.yml`

YAML anchor `x-backend` (строки 1–38) — не контейнер. Это переиспользуемый набор build-настройки и переменных для `backend` (строка 145: `<<: *backend`).
Конструкция `${NAME:?текст}` требует переменную; `${NAME:-default}` допускает отсутствие и подставляет default.

Строки	Разбор
1–38	Общие переменные backend: окружение, БД, CORS, уведомления, OAuth/SMTP и RAG. Секретные значения должны приходить только из `.env` /secret store, не из YAML.
40–60	`ml-service` : build из локального Dockerfile, healthcheck каждые 10 секунд, модель в named volume, порт доступен только localhost, лимиты CPU/RAM. `deploy.resources` реально учитывается Docker Compose, но поведение зависит от реализации Compose; проверяйте `docker compose config` .
62–85	`rag-agent` : аналогично; адрес backend берётся из Docker DNS-имени `backend` , а не из внешнего IP.
87–125	PostgreSQL 17. Данные сохраняются в `postgres_data` ; init SQL работает **только при первой инициализации пустого тома**. `pg_isready` — healthcheck. Параметры `-c` ограничивают соединения и запросы.
127–142	Swagger UI монтирует OpenAPI как read-only и ждёт backend только по порядку старта (`depends_on` без `condition`).
144–164	Backend получает shared settings, policy read-only, публикуется лишь на `127.0.0.1` , поэтому извне он недоступен напрямую. Ждёт здоровый Postgres и проверяет `/health` .
166–184	Демонстрационный provider уведомлений запускает тот же Go binary с аргументом `notify-demo` .
186–210	Dev frontend: исходники bind-mount, node_modules named volume, Vite; это удобно для разработки, но не подходит для production. Длинная команда в строке 200 чинит пустой/старый volume.
212–227	Prometheus: конфиг + правила монтируются read-only, данные остаются в named volume, порт только localhost.
229–243	`postgres-exporter` подключается к БД и отдаёт её метрики на 9187.
245–263	Alertmanager подставляет Telegram credentials в шаблон через entrypoint.
265–279	Loki хранит логи в volume и слушает localhost:3100.
280–293	Promtail читает Docker socket read-only для service discovery и отправляет логи в Loki. Docker socket — чувствительный доступ, ограничьте хост и доверенный образ.
295–317	Grafana получает credentials из `.env` , автоподключает источники/дашборды из `monitoring/grafana/provisioning` и работает за `/grafana/` .
319–341	Отдельный контейнер backup делает дамп PostgreSQL каждые 6 часов и хранит дневные/недельные/месячные копии на host bind mount `./backups` .

Строки	Разбор
343–350	Именованные Docker volumes: переживают пересоздание контейнера.
352–358	Docker network намеренно использует `172.28.0.0/16`, чтобы не пересечься с VPN хоста.

6. Сеть на Docker-хосте: Nginx

`nginx/default.conf` — **host Nginx**, не Nginx внутри frontend production image. Он проксирует запросы в localhost-порты Compose.

Строки 2–15: HTTP принимает ACME challenge Let's Encrypt, остальное перенаправляет на HTTPS.

18–87: fallback для прямого обращения по IP; защищает `.env/.git`, маршруты `/` → Vite 5173, `/api/` → backend 8080, `/docs/` → Swagger 8081, `/grafana` → Grafana 3001.

90–110: HTTPS `www` переадресовывает на канонический домен.
114–202 и 205–293: два TLS virtual host, сертификаты Let's Encrypt, TLS 1.2/1.3, заголовки безопасности и те же proxy rules.

`nginx/tuning.conf:2–5` создаёт корректный `Connection` для WebSocket upgrade; строки 7–35 включают gzip для текстовых ответов. Production K3s использует Traefik Ingress, так что это два разных входных контура, а не цепочка друг за другом.

7. Ansible и K3s

Inventory и playbooks

Файл/строки	Объяснение
`ansible/inventory.ini:1–9`	Группы `k3s_master`, `k3s_workers`, объединённая `k3s_cluster`. В inventory сейчас записаны реальный IP/путь SSH-ключа: для shared repo это плохая практика; вынести в незакоммиченный inventory или Ansible Vault.
`setup_k3s.yml:1–4`	Playbook повышает привилегии (`become`) и нацелен на `k3s_nodes`; такой группы в текущем inventory нет. Нужно добавить её в inventory или поменять `hosts` на существующую группу: `-l` может только сузить уже найденный набор, но не создаёт группу.
`:6–23`	Создаёт 2GiB swap, выставляет 0600, запускает `mkswap/swapon`. `ignore_errors` скрывает ошибку; также Kubernetes обычно требует отключённый swap или настройки kubelet, поэтому это надо перепроверить на конкретной версии K3s.
`:25–28`	Скачивает install script K3s и запускает его как root; `creates` делает задачу условно идемпотентной. Лучше закрепить версию/проверить checksum, а не исполнять pipe из сети.
`:30–34`	Забирает kubeconfig с сервера на controller. Этот файл содержит сертификат и клиентский private key — не коммитить.
`join_worker.yml:2–4`	Worker-only playbook.
`:7–8`	Адрес API и join token. Join token уже раскрыт в файле: его нужно немедленно ротировать и заменить на Vault/переменную CI.
`:11–28`	Повторяет swap setup.
`:30–33`	Устанавливает `k3s-agent`, передавая URL master и токен installer'у.
`install_fail2ban.yml:2–12`	На всех узлах ставит Fail2ban через APT и обновляет apt cache. Остальные строки записывают jail SSH и перезапускают сервис.
`ansible/k3s.yaml`	Локальный kubeconfig. `clusters` задаёт API/server+CA; `contexts` соединяет cluster, namespace и user; `users` содержит клиентский сертификат **и закрытый ключ**. Файл должен быть исключён из Git.

Модель K3s

Master/control-plane хранит API/планирует pods; worker выполняет назначенные

pods. Kubernetes Service даёт DNS (например, `http://postgres:5432`) независимо от IP pod'ов. Flannel в K3s создаёт overlay-сеть между нодами. `nodeSelector` в манифестах привязывает pod к ноде по label/hostname; это полезно для локального PV, но уменьшает отказоустойчивость.

8. Kubernetes без Helm: что означают манифесты `k8s/`

Любой YAML начинается с `apiVersion` (версия API), `kind` (тип объекта), `metadata` (имя/namespace/labels), `spec` (желаемое состояние). Label `app: backend` связывает Deployment → Pod → Service через selector.

Файл	Построчное чтение ключевых блоков
`backend-configmap.yaml:1–16`	`ConfigMap` в `queue-system`; строки 7–16 — несекретные ENV backend. `POSTGRES_HOST=postgres` — DNS Service. Нет `OTEL_EXPORTER_OTLP_ENDPOINT`, поэтому raw-манифест backend отключит трассировку, если не добавлена внешняя ENV.
`backend-deployment.yaml:1–55`	`Deployment`, 2 реплики; `RollingUpdate` 13–14 разрешает 1 дополнительный pod и запрещает недоступность; `selector` и template labels обязаны совпадать; контейнер читает ConfigMap+Secret; `requests` влияют на scheduler, `limits` — потолок; `/health` перезапускает pod при сбое, `/ready` исключает из Service.
`backend-service.yaml:1–15`	Внутренний `ClusterIP`, который на порту 8080 выбирает pods `app=backend`.
`frontend-deployment.yaml:1–52`	Те же rolling/probes для frontend; строки 23–24 жёстко выбирают одну hostname-ноду, образ `queue-frontend:v3`, порт 80, `/healthz`.
`frontend-service.yaml:1–15`	DNS `frontend`, ClusterIP 80 → container 80.
`ingress.yaml:1–25`	Traefik читает annotation; `/api` отправляется в backend:8080, `/` — во frontend:80. У правила нет `host`, значит оно подходит для всех host headers этого ingress.
`postgres-secret.yaml:1–10`	`stringData` позволяет записать текст, API кодирует его в `data`. Secret — не шифрование: base64 легко декодируется. Пароль в Git нужно заменить External Secret/Vault/SealedSecret.
`postgres-pvc.yaml:1–12`	PVC просит 2Gi, RWO (одна нода для записи), storage class K3s `local-path`.
`postgres-deployment.yaml:1–71`	Один pod + `Recreate`: безопаснее для RWO-диска, но даёт простой при обновлении. Строки 26–28 ENV из Secret; 36–57 проверяют `pg_isready`; 58–71 монтируют persistent data и ConfigMap init SQL. Init скрипт сработает лишь на новом data directory.
`postgres-service.yaml:1–15`	Внутреннее имя `postgres` и порт 5432.
`postgres-init-cm.yaml:1–385`	Сгенерированный ConfigMap: одна строка `data.init.sql` содержит объединённые SQL-миграции с `\n`. Его не нужно учить построчно как YAML: инфраструктурно важны `apiVersion`, `kind: ConfigMap`, `metadata.name: postgres-init` и mount в Deployment. Надёжнее запускать миграции отдельным Job, а не запекать SQL в ConfigMap.
`notification-demo-deployment.yaml:1–66`	Один demo pod, запускает образ с args `notify-demo`, публикует сервис 8082.

Файл	Построчное чтение ключевых блоков
`rag-agent-deployment.yaml:1–65`	Один RAG pod, `BACKEND_URL=http://backend:8080`, health probes; Service 8095. В ingress маршрута к нему нет.

9. Helm: «шаблоны + values = Kubernetes YAML»

`helm/queue-chart/Chart.yaml`:

Строки	Значение
1	Helm chart API v2.
2–3	Имя и описание пакета.
4	`application`, не library chart.
5	Версия **chart**, меняется при изменении шаблонов/values.
6	Версия приложения для людей/каталогов, не обязана быть Docker tag.

`values.yaml` — входные данные chart:

1–19 — образ, credentials, PVC и ресурсы Postgres.
21–45 — две backend-реплики, image tag, ENV, ресурсы и control-plane

`nodeSelector`.

47–61 — frontend image/2 реплики/resources.
63–69 — включённый Traefik ingress и host.

Шаблонные выражения нужно читать так: `{{ .Values.backend.image.tag }}` —

«подставь значение `backend.image.tag` из values»; `quote` добавляет YAML

кавычки; `toYaml | nindent 12` печатает map с правильным отступом;

`{{- … -}}` также удаляет пробелы вокруг шаблона.

Template, строки	Что генерируется
`templates/secret.yaml:1–9`	Postgres Secret из values. В текущем виде пароль всё ещё попадает в Git.
`templates/pvc.yaml:1–11`	PVC, с параметрами `storageClass` /size из values.
`templates/postgres.yaml:1–81`	Postgres Deployment/Service. Смысл тот же, что raw manifest, но image/resources/credentials параметризованы.
`templates/backend-configmap.yaml:1–16`	ConfigMap backend; важная добавка строки 16: OTLP endpoint Jaeger.
`templates/backend.yaml:1–68`	Deployment+Service backend; 8 — replicaCount, 22–25 — необязательный selector, 28 — image, 33–37 — config+secret, 40–53 — probes.
`templates/frontend.yaml:1–59`	Deployment+Service frontend; без nodeSelector в chart (в отличие от raw YAML).
`templates/ingress.yaml:1–73`	Условно создаётся при `ingress.enabled` ; цикл 10–42 строит rules на каждый host. Пути `/jaeger` и `/grafana` предполагают, что такие Service уже созданы отдельно; 43–72 добавляет ещё и host-less fallback rule.
`templates/postgres-init-cm.yaml`	Тот же сгенерированный SQL ConfigMap; Helm не превращает его в миграционный механизм.

Полезная безопасная проверка рендера: `helm template queue-release helm/queue-chart`

и затем `kubectl apply --dry-run=client -f <(helm template …)` .

10. Argo CD и GitOps

`k8s/argocd-app.yaml` буквально говорит Argo CD:

Строки	Значение
1–2	CRD Argo `Application`, это не стандартный Kubernetes kind.
4–7	Имя приложения в namespace Argo CD; finalizer удаляет управляемые ресурсы при удалении Application.
9	Argo project `default`.
10–16	Git repo, ветка/ревизия и папка Helm chart; `values.yaml` передаётся Helm.
17–19	Целевой API server — текущий кластер — и namespace workload.
20–25	Автосинхронизация: `selfHeal` исправляет ручной drift, `prune: false` **не удалит** ресурс, исчезнувший из Git; namespace создаётся автоматически.

`argocd-ingress.yaml:1–19` публикует UI Argo CD через Traefik по `/argocd`.

Сам Argo CD должен быть предварительно установлен (этот репозиторий не содержит его Helm install manifest). Рабочий цикл: изменить chart/values → commit/push в отслеживаемую revision → Argo получает Git → `helm template` → сравнивает live/desired → sync. Не надо параллельно вручную `kubectl edit` ресурсы Argo: `selfHeal` вернёт Git-версию.

11. Observability: метрики, логи и traces

Метрики: Prometheus → Alertmanager → Grafana

`backend/internal/httpapi/router.go:31–66` ставит middleware и открывает `/metrics` через Prometheus handler. В `middleware.go:254–295` определяются `http_requests_total{method,path,status}` и histogram `http_request_duration_seconds`; используется route pattern, а не ID в URL, поэтому не возникает dangerous high-cardinality label.

Файл, строки	Что делают
`monitoring/prometheus.yml:1–24`	Интервал scrape 15s; подгружает rules; направляет firing alerts на Alertmanager; собирает backend, postgres-exporter, себя и Alertmanager по Compose DNS.
`monitoring/prometheus/alerts.yml:1–56`	Группа правил: `BackendDown`, `PostgresDown`, exporter down, 5xx >5%, p95 >1s, соединения Postgres >80. `for` означает «условие должно длиться столько времени», чтобы не тревожить из-за одного scrape.
`alertmanager.template.yml:1–25`	Группирует alerts по имени/критичности, ждёт 10 s, повторяет через час, шлёт в Telegram и уведомляет о resolved. Placeholder подставляет shell entrypoint, а не Prometheus.
`entrypoint.sh:1–23`	`set -e`; если две Telegram ENV есть, `sed` строит конфиг во временном файле; иначе создаёт null receiver, затем `exec` заменяет shell процессом Alertmanager.
`k8s/observability/prometheus.yaml:1–83`	K8s-вариант ConfigMap + single replica Prometheus + Service. В нём интервал 10 s и retention 15d, но нет `rule_files`/Alertmanager — это функционально неполнее Compose-конфига.
`k8s/observability/postgres-exporter.yaml:1–53`	Exporter Deployment+Service. DSN записан открыто — заменить Secret.

Логи: slog → Promtail → Loki → Grafana

`backend/internal/observability/logging.go`:

11–34: `traceHandler` достаёт активный OTel span из context и добавляет `trace_id`/`span_id` в каждую JSON log запись.

36–42: логгер пишет JSON в stdout. Kubernetes/Docker забирает stdout без отдельного файлового агента в приложении.

Файл/строки	Что означает
`monitoring/promtail/promtail-config.yml:1–29`	Compose Promtail: хранит cursor в `/tmp`, использует Docker socket discovery, оставляет только текущий Compose project, превращает container/service в labels и добавляет `job=docker`.
`monitoring/loki/loki-config.yml:1–33`	Loki без auth, HTTP 3100, filesystem storage, single replica/in-memory ring, TSDB schema v13, ruler отправляет alerts в Alertmanager, analytics выключена. Здесь нет retention — она зависит от volume/настройки по умолчанию.
`k8s/observability/loki.yaml:1–118`	ConfigMap содержит похожий Loki config, но retention 168h. Deployment использует `emptyDir` для `/loki`: при пересоздании pod логи исчезнут. Для production нужен PVC/object storage.
`k8s/observability/promtail.yaml:1–150`	ServiceAccount → ClusterRole (read nodes/services/endpoints/pods) → binding. DaemonSet даёт по одному Promtail на ноду; hostPath `/var/log/pods` читает CRI logs. Pipeline парсит JSON slog, отбрасывает health и обычные GET, а `level/event/component` делает Loki labels. Обратная сторона: отброшенные логи уже не восстановить.

Traces: OpenTelemetry → Jaeger

`backend/internal/observability/tracing.go`:

Строки	Что происходит
21–26	`InitTracer` получает endpoint аргументом либо ENV `OTEL_EXPORTER_OTLP_ENDPOINT`.
28–35	Нет endpoint / `none` → noop provider, приложение продолжает работать без trace.
38–56	URL очищается от `http(s)://`; для HTTP endpoint включается insecure, TLS — нет. Таймаут exporter 5 секунд.
58–73	Создаётся OTLP/HTTP exporter и resource с `service.name`, окружением, host/process.
75–86	Batch processor и `AlwaysSample`; provider/propagator становятся глобальными. `TraceContext` переносит trace-id между сервисами через HTTP headers.
88–92	Лог и функция graceful shutdown, которая flush'ит spans.
`cmd/api/main.go:63–72`	Инициализация на старте; shutdown tracer вызывается с timeout 2s.
`httpapi/router.go:34`	`traceMiddleware` создаёт/продолжает server span HTTP-запроса; request log получает тот же context. Этот код сам по себе не доказывает трассировку всех downstream HTTP-вызовов или SQL: для них нужна отдельная instrumentation.
`k8s/observability/jaeger.yaml:1–76`	Jaeger all-in-one, включает OTLP; UI 16686, OTLP gRPC 4317, HTTP 4318; `QUERY_BASE_PATH=/jaeger` нужен для reverse proxy. Service публикует все три порта внутри кластера.
Helm ConfigMap:16	Задаёт backend endpoint `http://jaeger:4318`, т.е. именно Helm путь включает экспорт trace.

`AlwaysSample` годится для учебного стенда, но на нагруженном production обычно нужны parent-based / ratio sampling и защищённый OTLP endpoint.

`k8s/observability/grafana.yaml:1–122` provisioning-ом создаёт источники данных Prometheus/Loki/Jaeger (строки 7–25), file-provider dashboards (33–44), затем Deployment/Service Grafana. Строки 74–77 содержат жёстко заданные admin credentials — это нужно вынести в Secret и сменить.

12. Операционные скрипты

Файл	Для чего / ключевые строки
`scripts/release.sh`	Локальный release helper: 14–16 формирует tag из commit+времени, 23–26 запускает Go tests, 30+ собирает образ и импортирует его в containerd K3s, обновляет values и создаёт **локальный** commit. Скрипт лишь печатает предложение вручную `git push origin main`; он не делает push. Более того, Argo CD отслеживает `infra/k8s-gitops`, поэтому перед использованием надо согласовать ветку (иначе Argo commit не увидит). Это не GitLab CI.
`.gitlab-ci.yml`	Файл сейчас пустой: GitLab pipeline в этом репозитории фактически **не настроен**.
`scripts/autopull.sh`	Отдельный, не-Argo механизм периодически проверяет Git и перезапускает Compose. Не запускайте его для того же production, которым управляет Argo CD: получится два контроллера.
`scripts/install_autopull_service.sh:16–34`	Генерирует systemd service с циклом autopull, автоперезапуском и логами в `/var/log`. Скрипт изменяет `/etc`, запускать только осознанно с root.
`scripts/watchdog.sh`	Проверяет/рестартует Nginx и K3s, лечит Flannel FDB, проверяет Deployment replicas/external health, а при диске >88% удаляет образы. Последнее — потенциально разрушительное действие; требует ручной политики и backup-проверки.
`scripts/restore_backup.sh:19–27`	Запрашивает явное подтверждение и вливает `.sql.gz` в текущую БД — это перезаписывающая операция. Сначала проверить restore на отдельной БД.
`scripts/validate_env.sh:13–161`	Проверяет `.env`, required variables, порты, URL, LOG_LEVEL и ловит слабый dev password/localhost в production. `source .env` безопасен только для доверенного файла: shell выполнит его содержимое.
`scripts/optimize_server.sh`	Прямо меняет Docker daemon, sysctl и Nginx, чистит кэши. Это одноразовый root-скрипт для конкретного хоста, а не portable deployment manifest.

13. Что здесь требует исправления до production

1. **Секреты уже в Git.** Join token, kubeconfig/private key, пароли БД, Grafana/Argo credentials и DSN нужно считать скомпрометированными: rotate, удалить из истории по процедуре команды, использовать Vault/External Secrets или хотя бы Sealed Secrets. Никогда не считать Kubernetes Secret шифрованием. Адреса нод не являются секретами, но их лучше вынести из portable manifest в inventory/переменные окружения, чтобы не привязывать конфигурацию к одному провайдеру.
2. **Один delivery owner.** Выбрать: Compose + host Nginx + autopull *или* Argo CD + Helm + K3s. Для Kubernetes оставить Argo/Helm, отключить autopull на этих ресурсах.
3. **Свести конфигурации.** Raw K8s и Helm расходятся по tags, node selectors, CORS и OTEL. Для GitOps менять `values.yaml`, не raw YAML.
4. **Добавить полноценный production observability chart.** Сейчас Argo следит только за `helm/queue-chart`, а наблюдаемость находится в отдельных `k8s/observability/` и в chart не включена. Добавить их как Helm templates или отдельный Argo Application.
5. **Сделать данные устойчивыми.** PostgreSQL PVC уже есть, но Loki использует `emptyDir`; Jaeger all-in-one без persistent storage. Нужны PVC/object storage и backup/restore drills.
6. **Безопасность и supply chain.** Закрепить версии K3s/образов по digest, добавить image registry, сканирование образов, NetworkPolicy, RBAC с минимумом прав, TLS в ingress и resource quotas.
7. **Миграции.** Backend делает auto-migrate при старте двух реплик; это может конфликтовать при сложной схеме. Для production предпочтительнее отдельный Job/Argo sync hook до rollout.

14. Шпаргалка безопасной диагностики

```
# Compose
docker compose config                # итог после подстановки .env
docker compose ps
docker compose logs -f backend

# K3s/Kubernetes (на control-plane, namespace по умолчанию указан явно)
k3s kubectl get pods,svc,ingress -n queue-system
k3s kubectl get events -n queue-system --sort-by='.metadata.creationTimestamp'
k3s kubectl logs -n queue-system deployment/backend --tail=100 -f
k3s kubectl describe pod -n queue-system <pod-name>
k3s kubectl rollout status deployment/backend -n queue-system

# Helm / Argo CD
helm template queue-release helm/queue-chart
argocd app get queue-system           # если CLI авторизован

# Наблюдаемость
curl -fsS http://localhost:9090/-/ready
curl -fsS http://localhost:3100/ready
```

Не используйте `kubectl rollout restart` как первый ответ на проблему: сначала посмотрите events, `describe`, readiness, логи и метрики. Иначе перезапуск может скрыть первопричину.

15. Словарь

Image — неизменяемый шаблон файловой системы + команда запуска.
Container — запущенный экземпляр image.
Pod — минимальная единица запуска Kubernetes; обычно содержит контейнер приложения.

Deployment — поддерживает нужное число pod и rolling update.
Service — стабильное DNS-имя/виртуальный IP перед подами.
Ingress — HTTP-маршруты снаружи к Service; Traefik их исполняет.
ConfigMap / Secret — ENV/файлы конфигурации; Secret — лишь контроль

доступа/API-тип, не защита от утечки Git.

PVC — запрос на постоянное хранилище.
Probe — проверка liveness (перезапустить) или readiness (направлять ли трафик).

Metric / log / trace — число во времени / событие-текст / связанный путь одного запроса по компонентам.

GitOps — Git объявляет desired state, контроллер в кластере сам тянет и применяет изменения.

16. Helm с нуля: зачем он нужен именно здесь

16.1. Проблема, которую решает Helm

Без Helm для каждого окружения пришлось бы копировать один и тот же Deployment и менять вручную image tag, число реплик, память, домен и пароль. Например, минимальный **обычный Kubernetes YAML** для backend выглядел бы так:

```
# deployment.yaml — без Helm
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  selector:
    matchLabels: { app: backend }
  template:
    metadata:
      labels: { app: backend }
    spec:
      containers:
        - name: backend
          image: queue-backend:v3
          ports:
            - containerPort: 8080
```

Для staging нужно было бы создать почти идентичный файл, заменить `2` на `1`

и `v3` на другой tag. Helm оставляет единственный шаблон и выносит изменяемые

значения:

```
# values.yaml — входные данные
backend:
  replicaCount: 2
  image:
    repository: queue-backend
    tag: v3
```

```
# templates/backend.yaml — «программа», которая выдаёт YAML
spec:
  replicas: {{ .Values.backend.replicaCount }}
  template:
    spec:
      containers:
        - name: backend
          image: "{{ .Values.backend.image.repository }}:{{ .Values.backend.image.tag }}"
```

Команда `helm template` подставляет значения и возвращает первый обычный YAML.

Kubernetes не знает, что существовал Helm: он получает уже готовые

`Deployment`, `Service`, `ConfigMap` и другие объекты.

16.2. Термины, которые надо не путать

Термин	В этом проекте	Смысл
Helm	программа CLI и library	Рендерит/устанавливает chart.
Chart	`helm/queue-chart/`	Пакет шаблонов приложения.
Template	`helm/queue-chart/templates/backend.yaml`	Файл, содержащий YAML + `{{ … }}`.
Values	`helm/queue-chart/values.yaml`	Переменные для templates.
Release	например `queue-release`	Конкретная установленная копия chart с конкретными values.
Revision Helm	версия установки release	История Helm. Не то же самое, что Git commit и не то же самое, что ReplicaSet revision.
Argo Application	`queue-system`	Argo-объект, который указывает: из какого Git chart рендерить и куда синхронизировать.

Одна очень полезная формула для защиты:

```
Git commit → Argo Application → Helm chart + values → rendered manifests
→ Kubernetes Deployment → ReplicaSet → Pod → container
```

16.3. Как прочитать ваш chart от значения до pod

Возьмём backend образ.

1. В `helm/queue-chart/values.yaml` строки

21–28 определяют `backend.replicaCount: 2`, repository, tag и порт.

2. В `templates/backend.yaml`

строка 8 подставляет число в `spec.replicas` Deployment.

3. Строка 28 склеивает repository/tag в `image`, строка 31 делает порт

контейнера, 33–37 подставляют ConfigMap и Secret.

4. Kubernetes Deployment создаёт ReplicaSet с хешем в имени (на скриншоте

это `backend-5468bcc464`).

5. ReplicaSet создаёт два pod (`…-9qjhz` и `…-qzx6d` на скриншоте), потому

что desired replicas равны двум.

6. Service `backend` выбирает эти pod по label `app: backend`, а ingress

отправляет `/api` в Service, не в конкретное имя pod.

16.4. Helm-команды и безопасный порядок работы

Это команды для **изучения и проверки**, а не призыв заменить ими Argo CD на

production-кластере.

```
# 1. Проверить структуру chart и базовые ошибки
helm lint helm/queue-chart

# 2. Увидеть, какие YAML получится применить, ничего не меняя в кластере
helm template queue-release helm/queue-chart > /tmp/queue-rendered.yaml

# 3. Сравнить production и staging без копирования templates
helm template queue-release helm/queue-chart --set backend.replicaCount=1

# 4. Установить/обновить chart вручную — только для учебного/не-Argo окружения
helm upgrade --install queue-release helm/queue-chart \
  --namespace queue-system --create-namespace

# 5. Посмотреть Helm-release и его историю
helm list -n queue-system
helm history queue-release -n queue-system
helm get manifest queue-release -n queue-system
```

Почему не делать последний `upgrade --install` на управляемом Argo production:

Argo увидит ручное изменение как drift и при `selfHeal: true` может вернуть

Git-версию. Там корректное действие — изменить values в правильной Git-ветке,

сделать review/commit/push и дождаться Sync.

16.5. Полезные варианты values

Helm объединяет values: chart defaults → `-f файл` → `--set` ; последнее имеет

приоритет. Хорошая практика — не править главный values для каждого

окружения, а добавить (без секретов) `values-staging.yaml` :

```
# пример, файла пока в репозитории нет
backend:
  replicaCount: 1
  image:
    tag: "candidate-42"
frontend:
  replicaCount: 1
ingress:
  hosts: [staging.example.test]
```

Рендер: `helm template queue-release helm/queue-chart -f values-staging.yaml`.

Секреты не передают через `--set password=…`: они попадут в shell history и состояние release. Для production используют External Secrets/Vault/Sealed Secrets или созданный вне chart Secret с `existingSecret`-ссылкой.

16.6. Конкретные template-приёмы из вашего кода

```
# templates/backend.yaml, реальные выражения проекта
replicas: {{ .Values.backend.replicaCount }}     {{/* число из values */}}
image: "{{ .Values.backend.image.repository }}:{{ .Values.backend.image.tag }}"

{{- with .Values.backend.nodeSelector }}        {{/* если map не пуст */}}
nodeSelector:
  {{- toYaml . | nindent 8 }}                    {{/* map в YAML + отступ */}}
{{- end }}                                       {{/* иначе блока нет */}}
```

`with` меняет контекст `.` на содержимое `nodeSelector`; `$` в ingress template сохраняет ссылку на корневой context, поэтому `$.Values.backend.service.port` остаётся доступным внутри `range .Values.ingress.hosts`.

17. Что именно показывает ваш скриншот Argo CD

Скриншот показывает **Tree view Application** `queue-system`, а не «список запущенных контейнеров». Argo визуализирует зависимости Kubernetes-объектов.

На момент кадра у приложения два главных статуса:

Healthy — Argo считает текущие ресурсы здоровыми: нужные pods готовы, Deployment достиг желаемого состояния, PVC bound и т. п.

Synced to `infra/k8s-gitops` **(0797866)** — live-состояние соответствует желаемому состоянию из указанной Git revision. Это не означает «ошибок в коде нет» и не заменяет monitoring.

Auto sync is enabled — после commit в отслеживаемую ветку Argo сам синхронизирует chart. Статус `Sync OK` справа — результат последней sync.

17.1. Чтение дерева слева направо

```
Application queue-system
 ├─ postgres-pvc       (PersistentVolumeClaim: место для данных БД)
 ├─ postgres-secret    (Secret: параметры запуска PostgreSQL/backend)
 ├─ backend Service    (стабильное DNS-имя backend)
 ├─ frontend Service   (стабильное DNS-имя frontend)
 ├─ postgres Service
 ├─ backend Deployment
 │   ├─ ReplicaSet backend-5468bcc464  [текущий]
 │   │   ├─ Pod backend-…-9qjhz        [running 1/1]
 │   │   └─ Pod backend-…-qzx6d        [running 1/1]
 │   ├─ ReplicaSet backend-68f88bfc45  [старый]
 │   ├─ ReplicaSet backend-668b666b74  [старый]
 │   └─ …
 └─ frontend Deployment → ReplicaSets → Pods
```

Deployment не создаёт pod напрямую. Он создаёт ReplicaSet, а ReplicaSet создаёт pods. Хеш в имени ReplicaSet меняется, если меняется Pod template: image, ENV, labels, resources, probe и т. п. Это позволяет выполнить rolling update и откатиться к предыдущему шаблону.

17.2. Почему у некоторых ReplicaSet «0 pods»

Это нормальная картина после обновлений.

1. Был `backend-84fc69dc5b` с прежним образом/ENV и двумя pod.

2. Изменился template, например image tag. Deployment создал новый ReplicaSet с другим hash.

3. При `RollingUpdate`, установленном в chart (`maxSurge: 1`, `maxUnavailable: 0`), он сначала запускает новый pod сверх целевого числа, ждёт его readiness, затем постепенно уменьшает старый ReplicaSet.

4. Когда rollout закончен, у старых ReplicaSet `spec.replicas=0`, поэтому pod нет. Сами ReplicaSet остаются как **история rollout** для rollback.

На вашем кадре у `backend-5468bcc464` видно два running pod — это активный ReplicaSet. Записи `rev:1 … rev:5`, у которых нет pod, — старые ReplicaSet; Argo показывает их специально, чтобы была понятна история. Иконка/статус зелёный означает, что объект сам по себе корректен, а **не** что у него обязательно должен быть pod.

Это отличается от проблемного нуля:

Ситуация	Нормально?	Как отличить
Старый ReplicaSet после успешного rollout, replicas=0	Да	Есть новый RS с нужными ready pods; Deployment Healthy.
Deployment `replicas=2`, но готово 0/2	Нет	`kubectl describe deployment`, events, ImagePullBackOff/Pending/CrashLoopBackOff.
Service имеет 0 endpoints	Нет для работающего сервиса	Проверить labels selector и readiness pod.
Stateful PostgreSQL один pod	Да	В chart намеренно `replicas: 1`, PVC RWO.

Проверить доказательно, а не «по картинке»:

```
k3s kubectl get deployment,rs,pods -n queue-system -l app=backend
k3s kubectl get rs -n queue-system -o wide
k3s kubectl describe deployment backend -n queue-system
k3s kubectl get endpointslice -n queue-system -l kubernetes.io/service-name=backend
```

17.3. Почему на screenshot видны именно эти элементы

PVC создан chart template `pvc.yaml` и должен существовать до запуска PostgreSQL; он остаётся даже при пересоздании pod.

Secret создан `secret.yaml`, затем одновременно читается Postgres и backend через `envFrom`.

Service не имеет собственных pods: это виртуальная точка доступа, выбирающая pod по labels. Поэтому карточка `svc` не раскрывается как pod.

Deployment является контроллером, поэтому к нему привязаны ReplicaSet.
ReplicaSet — контроллер версии pod-template; «0 pods» у старого RS — сохранённая история, а не простаивающий сервис.

Pod `running 1/1` означает: в pod один container и он ready. `2/2`

означало бы два контейнера (например app + sidecar), не две реплики.

18. Мини-практика: от примера к этому репозиторию

Упражнение A — предсказать Helm render

Вопрос: если поменять в values:

```
backend:
  replicaCount: 3
  image:
    tag: v4
```

то что изменится? Правильный ответ: Helm сгенерирует Deployment с

`replicas: 3` и `image: queue-backend:v4` ; Service, имя pod и port не меняются.

Kubernetes создаст новый ReplicaSet из-за изменения image и постепенно

переведёт на него три pod. Старый RS в конце будет scale down до нуля.

Упражнение B — проследить конфигурацию OTEL

1. `templates/backend-configmap.yaml:16` задаёт

`OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4318` .

2. `templates/backend.yaml:33–35` загружает весь ConfigMap в ENV backend.
3. `cmd/api/main.go:63` вызывает `observability.InitTracer` .
4. `tracing.go:24–26` берёт endpoint из ENV.
5. `tracing.go:58` создаёт OTLP/HTTP exporter.

6. `k8s/observability/jaeger.yaml:74–76` публикует Service port 4318.

Итог: это работающий путь **только если** Jaeger Service реально применён в том же namespace. Потому и важен вопрос комиссии: «Кто деплоит observability resources, если Argo Application указывает лишь на `helm/queue-chart`?»
Честный ответ: сейчас это отдельные raw manifests; их нужно либо применить отдельной Argo Application, либо перенести в templates chart.

Упражнение C — почему Service находит новые pod после rollout

```
# из Service backend
selector:
  app: backend

# из pod template Deployment
labels:
  app: backend
```

Service не знает hash ReplicaSet и имена pod. Пока label совпадает и readiness успешна, новый pod автоматически попадает в EndpointSlice Service. Поэтому можно заменить pod без смены URL `http://backend:8080`.

19. План живой демонстрации на защите (5–7 минут)

1. Откройте Argo CD и скажите: «Это GitOps controller. Healthy — ресурсы готовы; Synced — они совпадают с Git revision, а не просто с тем, что я нажал вручную».
2. Покажите `values.yaml`: «В этом файле меняются параметры release: tag, replicas, resources; шаблоны не дублируются для каждого окружения».

3. Откройте `templates/backend.yaml`, покажите строки `replicas`, `image`, `envFrom`, probes. Проговорите цепочку values → template → Deployment → RS → Pod.

4. Вернитесь к дереву: «0 pod у старых ReplicaSet — нормальная история rollout. Активный RS имеет текущие ready pod; service выбирает их по label». Не утверждайте это без команды `kubectl get rs`.

5. Откройте `backend-configmap.yaml` Helm и `tracing.go`: «ENV Jaeger → InitTracer → OTLP/HTTP → Jaeger; JSON logs несут trace_id, Prometheus берёт `/metrics`».

6. Закончите честным improvement: «Я бы убрал секреты из Git, вынес observability в тот же GitOps контур, добавил persistent storage Loki и отделил DB migrations в Job».

20. Вопросы комиссии и короткие сильные ответы

Вопрос	Ответ
Зачем Kubernetes, если есть Compose?	Compose хорошо запускает набор контейнеров на одном host. Kubernetes добавляет scheduler, desired state, rolling update, service discovery, probes и multi-node модель.
Что делает Helm?	Рендерит параметризованные Kubernetes YAML из templates и values. Helm не заменяет Kubernetes и не запускает container сам.
Helm и Argo CD — одно и то же?	Нет. Helm — packaging/template engine. Argo CD — GitOps controller, который в данном проекте вызывает Helm render и синхронизирует результат с кластером.
Что является source of truth?	Для GitOps — chart/values **в revision, указанной Argo Application**. Для локальной среды — Compose+ `.env` . Raw `k8s/` нельзя считать source of truth одновременно с chart.
Почему старые ReplicaSet не удалены?	Это revision history Deployment для rollback. После rollout их replicas=0, поэтому pods нет. Историю ограничивают `revisionHistoryLimit` , если она становится слишком большой.
Почему Service не создаёт pod?	Service — стабильный виртуальный адрес и selector, controller'ом pod является Deployment/ReplicaSet.
Чем liveness отличается от readiness?	Liveness failure перезапускает container. Readiness failure не перезапускает, но убирает pod из endpoints Service до готовности.
Почему PostgreSQL один pod?	Это stateful сервис с RWO disk; текущий chart выбирает простоту. Высокая доступность БД потребует StatefulSet, replication/failover и другой storage design.
Что произойдёт при изменении tag образа?	Изменится pod template → новый ReplicaSet → rolling update согласно maxSurge/maxUnavailable → старый scale down. `IfNotPresent` означает, что нужный tag уже должен быть доступен на node/в registry.
Почему `:latest` плох?	Tag изменяемый, сложно понять что реально запущено и откатиться. Лучше immutable tag или image digest.
Что такое ConfigMap и Secret?	Оба передают config в pod; Secret — отдельный API-object/access policy, но `stringData` /base64 не защищают секрет, если он лежит в Git.
Что делает `envFrom` ?	Импортирует все keys ConfigMap/Secret как ENV контейнера. Это удобно, но скрывает список зависимостей; для критичных values иногда лучше явный `env` .
Почему `/ready` проверяет БД?	Backend без БД не может обслужить бизнес-запрос; kube-proxy не должен отправлять ему трафик.
Что такое OTEL trace?	Связанный набор span одного запроса. Trace ID общий, у каждого span свой span ID и timing/attributes.

Вопрос	Ответ
Что показывает Prometheus, а что Loki?	Prometheus — агрегируемые числовые time series и alerts; Loki — дискретные logs и поиск по labels/тексту.
Для чего Jaeger?	Посмотреть задержки и ошибки по участкам одной трассы. Нужна instrumentation каждого интересующего вызова, одного server span недостаточно для SQL/downstream детализации.
Почему Promtail DaemonSet?	Нужен агент на каждой ноде, чтобы читать node-local pod logs. Deployment дал бы лишь несколько произвольных агентов и потерял логи других nodes.
Что делает Ansible?	Идемпотентно описывает настройку хостов по SSH: пакеты, файлы, K3s agent/server. Он не является Kubernetes deployment engine.
Чем `kubectl apply` хуже в GitOps?	Не хуже как инструмент, но ручное применение обходит review/audit Git и создаёт drift. В этом проекте `selfHeal` Argo может его отменить.
Что проверите при CrashLoopBackOff?	`kubectl describe pod` и events, затем `kubectl logs --previous`; ENV/Secret, command, probes, port, image tag и resource limit. Не начну с restart.
Что проверите при Pending?	`describe pod`: обычно нет ресурсов, nodeSelector/taint, PVC не bound или image scheduling constraint.
Что улучшите первым?	Ротация раскрытых секретов и вынос из Git; затем один GitOps owner и включение observability в тот же Argo flow.

21. Чего не стоит утверждать на защите

Не говорите «Kubernetes гарантирует отсутствие простоя»: текущие backend settings (`maxUnavailable: 0`) помогают, но реальные гарантии зависят от ресурсов, readiness и доступности базы/ingress.

Не говорите «Secret зашифрован»: в manifest `stringData` , а Kubernetes Secret обычно base64; защита зависит от RBAC, encryption at rest и secret manager.

Не говорите «Argo построит Docker image»: Argo CD в этой схеме рендерит и применяет manifests; сборка/push/import image — другой шаг CI/release.

Не говорите «все логи есть в Loki»: K8s Promtail специально отбрасывает health и routine GET, а Loki на `emptyDir` теряет данные после пересоздания.

Не говорите «все вызовы уже trace’ятся»: доказан inbound HTTP span и export;

downstream/SQL instrumentation нужно проверить отдельно.

22. Переход Docker Compose → Kubernetes: правильная ментальная модель

22.1. Короткий ответ

**Нет, pod — не копия `docker-compose.yml`, и Kubernetes не запускает Docker

Compose внутри каждой pod.** Compose и Kubernetes — два разных оркестратора.

Docker Compose	Kubernetes / K3s
docker-compose.yml	Helm templates → K8s manifests
Docker Engine	container runtime (в K3s — containerd)
service: backend	Deployment backend
container backend	Pod backend → container backend
Compose network + DNS	Service backend + cluster DNS
depends_on + healthcheck	readiness/liveness probes + scheduler
docker compose up	Argo CD / kubectl / Helm reconciliation

Контейнеры остаются как базовый способ запустить процесс: backend всё ещё

работает в image `queue-backend:<tag>`, frontend — в `queue-frontend:<tag>`.

Меняется тот, кто их создаёт и поддерживает:

Compose создаёт контейнеры Docker на одной машине на основании одного YAML.
Kubernetes создаёт Pod из `Deployment`; внутри Pod запускает один или

несколько OCI-контейнеров runtime’ом `containerd` (у K3s по умолчанию).

Docker CLI может вообще не быть установлен на Kubernetes-ноде. Команды

`docker ps` там не обязаны показать Kubernetes Pod; вместо них используют

`kubectl get pods` или `k3s crictl ps`.

Если Docker Compose продолжает быть запущенным на том же сервере, его контейнеры **физически останутся**, пока их явно не остановить:

```
# Только проверить Compose-сервисы; ничего не останавливает
docker compose ps

# Посмотреть Kubernetes workload
k3s kubectl get pods -A
```

Это две независимые группы процессов. Останавливать `docker compose down` можно только после того, как новый K8s-путь проверен, трафик переведён и есть backup; иначе можно выключить работающий production.

22.2. Почему «одна pod = весь Compose» неверно

В Compose project — это группа сервисов. В Kubernetes разбиение явное:

```
docker-compose.yml                    Kubernetes-объекты
──────────────────────────────────    ──────────────────────────────────────────
backend service                  →    Deployment backend (2 pod) + Service backend
frontend service                 →    Deployment frontend (2 pod) + Service frontend
postgres service                 →    Deployment/лучше StatefulSet postgres + Service +
prometheus service               →    отдельный Deployment prometheus + Service
promtail service                 →    DaemonSet promtail (по одному на node)
postgres-exporter service        →    Deployment exporter + Service
network default                  →    cluster network + DNS + Service selectors
volumes postgres_data            →    PersistentVolumeClaim postgres-pvc
```

Поэтому на node может находиться только часть приложения. Scheduler решает, где запускать pod, с учётом requests, limits, nodeSelector, taints, PVC и доступности ноды. В текущем проекте часть манифестов жёстко закрепляет

workload по `kubernetes.io/hostname` или `control-plane` label; это ограничивает scheduler и делает текущую схему ближе к «нескольким специально назначенным серверам», чем к полностью динамичному кластеру.

22.3. Что переехало в Kubernetes в этом репозитории, а что нет

Компонент	Есть в Compose	Есть в raw `k8s/`	Есть в Helm chart, который применяет Argo	Вывод
Backend	Да	Да	Да	Полностью описан двумя альтернативными способами K8s.
Frontend	Да	Да	Да	То же.
PostgreSQL + PVC	Да	Да	Да	В K8s есть диск/PVC, но это не HA БД.
Ingress	Host Nginx	Да	Да	Compose и K8s используют разные входные контуры.
Prometheus/Loki/Promtail/Grafana/Jaeger	Да (Jaeger отсутствует в Compose)	Да	Нет, кроме маршрутов Ingress к Jaeger/Grafana	Их надо включить отдельным Argo Application или перенести в chart.
Alertmanager / backup	Да	Нет	Нет	При переходе на чистый K8s их нельзя «считать переехавшими».
ML service / RAG agent / notification demo	Да	Частично	Нет	Raw manifests есть для RAG/demo, но Helm/Argo их не создаёт; ML-service вообще не имеет K8s manifest.
Swagger	Да	Нет	Нет	В K8s-пути не описан.

Из этого следует честный ответ для защиты: **в проекте есть гибридный период миграции, а не один законченный бесшовный deployment definition**. Argo на скриншоте управляет тем, что рендерит `helm/queue-chart`; остальное нужно

проверять по отдельным Application/ `kubectl get` .

22.4. Как один сервис «поднимается» в Kubernetes

Возьмём backend и пройдём цепочку без магии:

```
1. Docker build / CI строит queue-backend:<immutable-tag>
2. Image попадает в registry, доступный каждой K8s node
3. Git меняет backend.image.tag в Helm values
4. Argo CD замечает новый Git commit
5. Argo CD запускает Helm render и отправляет Deployment в Kubernetes API
6. Deployment controller создаёт новый ReplicaSet
7. ReplicaSet просит scheduler создать Pod
8. Scheduler выбирает node
9. kubelet выбранной node говорит containerd скачать image и запустить container
10. /health и /ready проходят; EndpointSlice добавляет pod к Service backend
11. Traefik начинает направлять новые /api запросы в готовый pod
12. Старый ReplicaSet scale down; его Pod завершаются grace period'ом
```

В Compose шаги 3–11 заменяются одной локальной командой `docker compose up -d

--build`; нет API-server, scheduler, ReplicaSet, Pod, cluster Service или

desired-state reconciliation.

22.5. Важный нюанс про images в текущей реализации

`scripts/release.sh` делает `docker save | ssh … k3s ctr images import -` на

один `MASTER_HOST` . Это может работать, когда backend всегда назначен на master

через `nodeSelector` , как в Helm values. Но если pod сможет попасть на worker,

на worker image может отсутствовать, и появится `ImagePullBackOff` .

Production-подход масштабируется так:

```
CI builds image → push registry.example/queue-backend@sha256:…
→ Helm values stores immutable tag/digest → every node can pull it
→ scheduler may safely place pod on any eligible node
```

Для защиты: не надо утверждать, что текущий `ctr import` — полноценный image registry. Это учебный/малый-cluster workaround; registry решает distribution, а CI добавляет scan, provenance и повторяемость build.

23. Как DevOps проектируют такой проект: от простого к сложному

Ниже не «единственный правильный порядок», а практичный порядок зависимостей. Нельзя развернуть приложение в кластере, пока нет нод и сети; нельзя полезно строить алерты, пока неизвестны health/SLO и приложение не отдаёт метрики.

```
0. Требования и границы
1. Код приложения, контейнеры, локальная среда
2. Базовая инфраструктура серверов
3. Kubernetes-кластер и доступ к нему
4. Registry, namespaces, policy и storage
5. Platform-слой: ingress, certs, secrets, GitOps
6. Application manifests/Helm и миграции
7. Observability: metrics/logs/traces/alerts
8. CI/CD и release process
9. Надёжность: backup, restore drill, scaling, security, runbooks
```

Шаг 0. Сформулировать эксплуатационные требования

До выбора Helm или Kubernetes DevOps вместе с командой фиксирует:

какие компоненты есть: frontend, API, БД, внешние integrations, background

workers, ML/RAG;

SLO: например доступность API, p95 latency, RPO/RTO базы;
предполагаемую нагрузку, бюджет CPU/RAM/disk;
где будут храниться secrets, образы, бэкапы и логи;
окружения (local/dev/staging/prod), домены/TLS и кто имеет доступ;
как делать rollback и кто реагирует на alert.

Это объясняет, **почему** в проекте есть `resources`, probes, backup, Loki и Alertmanager, а не просто набор YAML «для галочки».

Шаг 1. Сначала сделать приложение пригодным для запуска

Разработчик и DevOps совместно задают договор контейнера:

```
image → command → port → ENV → persistent data → health/readiness → logs → metrics
```

В вашем backend это уже видно:

Нужда платформы	Реализация в проекте
Предсказуемый процесс	`backend/Dockerfile`, `ENTRYPOINT ["/app/api"]`.
Настройка без пересборки	`config.Load()` читает ENV; K8s даёт их через ConfigMap/Secret.
Готовность/живость	`router.go` реализует `/health`, `/ready`; Deployment использует их в probes.
Корректное завершение	`main.go` слушает SIGTERM/SIGINT и вызывает server shutdown. Kubernetes посылает SIGTERM при rollout.
Логи	`slog` пишет JSON stdout; Promtail читает stdout через node logs.
Метрики	`promhttp.Handler()` выдаёт `/metrics`; Prometheus scrape'ит endpoint.
Трейсы	`InitTracer` и middleware создают/экспортируют spans.

Именно поэтому observability встраивают **в код рано**, вместе с endpoint здоровья и graceful shutdown, а инфраструктуру сбора/хранения можно подключить следующим шагом. Иначе после инцидента будет нечем доказательно диагностировать проблему.

Шаг 2. Собрать и проверить локальную среду Compose

Compose — полезная нижняя ступень, не «устаревшая ошибка»:

```
./scripts/validate_env.sh .env       # проверка конфигурации
docker compose up --build -d         # собрать и запустить сервисы
docker compose ps                    # статус контейнеров
docker compose logs -f backend       # посмотреть JSON-логи
curl -fsS http://127.0.0.1:<HTTP_PORT>/ready
```

Здесь проверяют: image действительно собирается, ENV достаточны, API видит БД, ports не конфликтуют, logs/metrics существуют. Compose остаётся для локальной разработки, интеграционных тестов и небольшого single-host deployment даже после появления K8s; он не обязан «исчезнуть из репозитория».

Шаг 3. Подготовить серверы Ansible-ом

Ansible работает **до Kubernetes workload**, потому что ему нужны SSH-доступ, ОС и ноды. Типовая последовательность:

```
VM/servers provisioned
→ SSH keys, users, firewall, update OS
→ Ansible inventory/group vars/Vault
→ install K3s server на control-plane
→ забрать/настроить kubeconfig
→ install K3s agents на workers
→ проверить nodes и cluster networking
```

Команды с текущими файлами требуют сначала исправить `hosts: k3s_nodes` в `ansible/setup_k3s.yml` или добавить такую inventory group:

```
ansible-inventory -i ansible/inventory.ini --graph
ansible-playbook -i ansible/inventory.ini ansible/install_fail2ban.yml
# После исправления inventory/playbook target:
ansible-playbook -i ansible/inventory.ini ansible/setup_k3s.yml
ansible-playbook -i ansible/inventory.ini ansible/join_worker.yml
```

Проверка кластера:

```
export KUBECONFIG=/path/to/private/k3s.yaml
kubectl get nodes -o wide
kubectl get pods -A
kubectl cluster-info
```

Нельзя безопасно копировать literal join token/kubeconfig из этого репозитория: они раскрыты и подлежат ротации. В нормальном проекте это Ansible Vault или секрет CI, а не `vars` playbook.

Шаг 4. Развернуть базовый platform layer

До application обычно создают:

1. **Namespace** — логическая граница (`queue-system`).

2. **StorageClass/PVC policy** — где Postgres/Loki будут хранить данные.

3. **Ingress controller** — в K3s обычно уже включён Traefik; проверить,

а не предполагать: `kubectl get pods -n kube-system` .

4. **TLS/cert-manager** и DNS — чтобы ingress был HTTPS, а не только HTTP.

5. **Registry** — доступный всем nodes, и `imagePullSecrets` если нужен.

6. **Secrets policy** — Vault/External Secrets/Sealed Secrets и минимальный

RBAC.

7. **ResourceQuota/LimitRange/NetworkPolicy** — когда есть несколько команд

или production risk.

В текущем chart namespace может создать Argo (`CreateNamespace=true`),

`local-path` storage использован для PostgreSQL, а Traefik указан через

annotation. cert-manager, registry, NetworkPolicy, quotas и внешнее secret

management пока не описаны в репозитории.

Шаг 5. Поставить Argo CD и описать GitOps

Это platform component, устанавливаемый **до** приложения. Типовая схема:

```
# пример команд; версия/репозиторий выбираются командой и фиксируются в IaC
kubectl create namespace argocd
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update
helm upgrade --install argocd argo/argo-cd -n argocd

# Затем применяют только Argo Application из Git
kubectl apply -f k8s/argocd-app.yaml
```

В результате Argo получает ограниченный доступ к Git и Kubernetes API. Его Application указывает repo/revision/path/destination. Для production обычно добавляют `AppProject`, ограничивающий allowed repos/namespaces/kinds, SSO/RBAC и защищённый HTTPS ingress, а не используют `project: default`.

Шаг 6. Описать application в Helm, затем развернуть

Сначала проверяют chart локально:

```
helm lint helm/queue-chart
helm template queue-release helm/queue-chart > /tmp/queue.yaml
kubectl apply --dry-run=client -f /tmp/queue.yaml
```

Далее GitOps-путь:

```
build/push immutable image
→ change values image tag in Argo-tracked branch
→ pull request/review
→ merge/push
→ Argo refresh/sync
→ kubectl checks + application smoke test
```

В этом проекте надо явно устранить расхождение: Argo Application следит за `infra/k8s-gitops`, а `release.sh` предлагает `git push origin main`. Без единой согласованной ветки commit может быть успешным, но deployment не изменится.

Шаг 7. Добавить наблюдаемость как часть платформы, а не «после аварии»

Логический порядок такой:

```
код отдаёт /health, /ready, /metrics, JSON logs, OTEL spans
→ collectors/storage (Prometheus, Promtail, Loki, Jaeger)
→ Grafana dashboards
→ alert rules + Alertmanager route
→ runbook: кто, что и за сколько делает при alert
```

В этом проекте прикладная часть уже реализует health/readiness/metrics/traces, а K8s manifests для collectors есть в `k8s/observability`. Но они пока не входят в Helm chart, который синхронизирует Argo. Следующий логичный шаг — отдельный `Application` для observability или перенос manifests в chart; так monitoring получает тот же audit trail/reconciliation, что и приложение.

Шаг 8. Построить CI, release и rollback

Полноценный pipeline отличается от текущего локального `release.sh`:

```
commit → unit/integration tests → build image → vulnerability scan
→ push immutable image to registry → update GitOps values
→ Argo deploys → smoke test / metrics check → promote/rollback
```

`.gitlab-ci.yml` пуст, поэтому GitLab CI ещё не выполняет эту цепочку. На защите корректно назвать `release.sh` временным release helper, а не полноценным CI/CD. Rollback обычно означает вернуть Git commit/tag и дождаться Argo sync; `kubectl rollout undo` полезен аварийно, но после него Git должен быть приведён к тому же desired state.

Шаг 9. Усложнять только после базовой работоспособности

Когда базовый путь стабилен, добавляют HPA/VPA, PDB, multi-zone storage, Postgres HA, canary/blue-green, policy-as-code, cost monitoring, disaster recovery exercises. Это нельзя достоверно «получить» одним YAML — каждое усложнение требует метрик, тестов отказа и runbook.

24. Полная логика проекта как рассказ на защите

Вот последовательный рассказ, который связывает все инструменты без прыжков:

1. «У нас Go API, React frontend и PostgreSQL. Я сначала сделал каждый компонент контейнеризуемым: Dockerfile даёт повторяемый image, ENV — конфигурацию, `/health` / `/ready` — контракт с оркестратором».
2. «Для разработки и проверки всего набора на одной машине использую Compose. Он создаёт Docker containers и сеть, но не даёт multi-node scheduling и desired-state controller».
3. «Для серверов Ansible по SSH готовит OS/ноды и устанавливает K3s; K3s даёт Kubernetes API, scheduler, containerd runtime и сетевую модель кластера».
4. «В Kubernetes я не переношу Compose как один pod. Backend, frontend и DB — самостоятельные workload; Service даёт стабильную связь, PVC — постоянные данные, Ingress/Traefik — внешний HTTP».
5. «Helm убирает копирование YAML: values хранят параметры, templates создают Deployment/Service. Изменение image tag создаёт новый ReplicaSet и rolling update, поэтому старые RS с 0 pod в Argo — ожидаемая history».
6. «Argo CD берёт этот Helm chart из Git и синхронизирует desired state, что делает Git audit trail. Ручной `kubectl edit` не является обычным путём».
7. «Наблюдаемость проектировалась контрактом приложения: Prometheus metrics,

JSON logs → Loki, OTel spans → Jaeger, Grafana объединяет просмотр; alerts идут через Alertmanager».

8. «Дальнейшие улучшения: удалить secrets из Git, единый registry/CI, сделать monitoring частью Argo, persistent storage observability и отдельный Job миграций».

Если вас спросят «почему не сразу Kubernetes?», ответ: «Compose быстрее отлаживает локальный комплект и остаётся dev-инструментом. Kubernetes оправдан, когда нужны управляемые rollout, self-healing, стабильная сервисная сеть и несколько нод. Мы не дублируем Compose в pod, мы переводим каждый сервис в подходящий Kubernetes workload».

25. Kubernetes без магии: 12 основ, которые должен знать новичок

25.1. Desired state и controllers

В Kubernetes обычно не пишут «запусти container прямо сейчас». Вместо этого пишут desired state: «должны существовать два ready pod backend с таким image». Controller постоянно сравнивает желаемое (`spec`) и фактическое (`status`):

```
spec.replicas = 2, а фактически один pod жив
→ Deployment controller создаёт/просит ReplicaSet
→ ReplicaSet создаёт второй Pod
→ scheduler назначает node
→ kubelet запускает container
→ status.readyReplicas становится 2
```

Это и называется **reconciliation loop**. Self-healing — не магическое свойство контейнера: если pod умер, controller замечает расхождение и создаёт

новый. Данные в `emptyDir` при этом потеряются; состояние должно жить во внешней БД/PVC/object storage.

25.2. Namespace — не node и не окружение автоматически

Namespace группирует API-объекты и ограничивает их scope. `queue-system` в этом проекте — namespace приложения, `argocd` — namespace GitOps-controller.

```
# Одна и та же команда с разными scope
kubectl get pods -n queue-system
kubectl get pods -n argocd
kubectl get pods -A                 # все namespaces
kubectl config set-context --current --namespace=queue-system
```

`namespace=staging` сам по себе не создаёт отдельный кластер, DNS, secrets и database. Изоляцию дополняют отдельными values, RBAC, quotas, NetworkPolicy и, иногда, отдельным кластером.

25.3. Labels, selectors и annotations

```
metadata:
  labels:
    app: backend                # короткое индексируемое свойство
    app.kubernetes.io/version: v3
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: web,websecure
```

Label — короткое поле для отбора/группировки. Service выбирает pods по label; `kubectl get pods -l app=backend` использует тот же механизм.

Selector — запрос к labels. Если Service selector не совпал с Pod labels,

Service будет иметь ноль endpoints, хотя pod может быть `Running`.

Annotation — не предназначена для selector, это произвольная метаинформация для controller/инструмента. В project Traefik читает annotation Ingress.

25.4. Pod, container и sidecar

Pod — единица scheduling: контейнеры внутри одного pod разделяют сеть (`localhost`), IP и volumes. Не обязательно один pod = один контейнер.

```
Pod backend
 ├─ app container        :8080
 └─ (возможный) sidecar  : например proxy/agent
```

В текущем backend pod один container, поэтому `1/1`. Sidecar для логов в этом проекте не нужен: Promtail как DaemonSet читает node-level stdout logs. Не добавляйте sidecar «потому что так принято» — он увеличивает ресурсы, probes и сложность shutdown.

25.5. Service и DNS

Pod IP временный. Pod после restart получит новое имя/IP. Service создаёт постоянное виртуальное имя и выбирает готовые endpoints:

```
backend pod IPs: 10.42.1.18, 10.42.2.91 (могут меняться)
Service DNS:    backend.queue-system.svc.cluster.local
Короткое имя:   backend          (из pod того же namespace)
```

Поэтому ConfigMap задаёт `POSTGRES_HOST: postgres`, а RAG —

`BACKEND_URL=http://backend:8080`: это cluster DNS, не хардкод IP. `ClusterIP` доступен внутри кластера. Внешний трафик приходит через Ingress/LoadBalancer, а не через pod IP.

25.6. Ingress и Traefik

Ingress — правило маршрутизации HTTP(S), сам YAML не открывает порт. Его исполняет ingress controller; в K3s это обычно Traefik.

```
Internet → DNS → node:80/443 → Traefik
  /api/* → Service backend:8080 → ready backend Pod
  /*     → Service frontend:80  → ready frontend Pod
```

Порядок маршрутов, `pathType`, host и TLS имеют значение. `pathType: Prefix` в проекте означает, что `/api` покрывает `/api/...`. Для реального домена нужны DNS record, TLS certificate и ingress-controller; одного файла Ingress недостаточно.

25.7. Scheduler, requests, limits, taints, nodeSelector

```
resources:
  requests:
    cpu: 100m       # scheduler резервирует 0.1 CPU
    memory: 64Mi    # scheduler учитывает минимум RAM
  limits:
    cpu: 500m       # потолок CPU: throttling при превышении
    memory: 256Mi   # превышение обычно приводит к OOMKilled
```

`m` у CPU означает millicores: `100m = 0.1` CPU. `Mi` — mebibytes.

Requests помогают scheduler решить, поместится ли pod; limits защищают node от одного прожорливого container. Неверные маленькие requests дают Pending/OOM, а слишком большие — пустой дорогой кластер. Их выбирают по измерениям.

`nodeSelector` в chart backend требует label control-plane. Это **жёсткое условие**: scheduler не разместит pod на worker без этого label. Taint отталкивает pods от node, toleration разрешает игнорировать конкретный taint. В Promtail DaemonSet есть tolerations, поэтому он работает и на control-plane.

25.8. Liveness, readiness и startup probe

Проверка	Вопрос	При ошибке	В вашем проекте
livenessProbe	«Процесс безнадёжно завис?»	kubelet перезапускает container	backend `/health`; frontend `/healthz`.
readinessProbe	«Можно ли давать трафик?»	Pod исключается из Service endpoints	backend `/ready` проверяет БД.
startupProbe	«Долго ли приложение стартует?»	Пока она не прошла, liveness/readiness не мешают запуску	Сейчас нет; полезна для медленных migration/cold start.

Не делайте liveness проверкой удалённой БД: временный сбой БД начнёт бесконечно перезапускать здоровые API pod. Для backend корректно разделить лёгкий `/health` и зависимый `/ready`, как сделано здесь.

25.9. Deployment, StatefulSet, DaemonSet, Job и CronJob

Workload	Когда выбирать	Пример из/для проекта
Deployment	Stateless API/UI, взаимозаменяемые replicas	backend, frontend, Grafana, Jaeger.
StatefulSet	Stable identity/storage/упорядоченный lifecycle	Лучше подходит для production PostgreSQL, чем Deployment.
DaemonSet	Один pod на каждую eligible node	Promtail.
Job	Одноразовая работа до успешного завершения	DB migration, data import.
CronJob	Job по расписанию	backup PostgreSQL.

Текущий Postgres описан Deployment + PVC — допустимо для учебного singleton, но не превращает его в HA БД. Compose backup-container при переносе в K8s логичнее стать CronJob с Secret/PVC/object-storage target.

25.10. Volumes: ephemeral и persistent

Тип	Живёт сколько	Пример
container filesystem	до пересоздания container	временные файлы приложения.
`emptyDir`	до удаления Pod	positions Promtail, текущий Loki storage — поэтому логи теряются.
ConfigMap/Secret volume	управляется API, read-only по смыслу	config/credentials, init.sql.
PVC/PV	независимо от pod (по policy storage)	Postgres data.

PVC — запрос на storage, PV — выделенный actual volume. `ReadWriteOnce` не значит «ровно один pod всегда»: обычно это запрет одновременного mount с записью несколькими node. Semantics зависят от storage provider. Перед удалением PVC обязательно понять `reclaimPolicy` и сделать проверенный backup.

25.11. API version, manifest и server-side truth

YAML в Git — desired state, но Kubernetes API добавляет defaults/status и может изменять объект admission controllers. Поэтому полезны три взгляда:

```
# desired render до кластера
helm template queue-release helm/queue-chart

# live object и status из API server
kubectl get deployment backend -n queue-system -o yaml

# человекочитаемые причина/Events
kubectl describe deployment backend -n queue-system
```

`kubectl apply --dry-run=client` ловит часть YAML ошибок без кластера; `--dry-run=server` проверяет также API/admission, но требует доступ к кластеру.

25.12. Почему не нужно всегда делать `kubectl delete pod`

Удаление pod может временно заставить controller пересоздать его, но это не лечит неверный Secret, image, limit или DNS. При Stateful service может ещё и усложнить восстановление. Привычка инженера: сначала `get`, `describe`, events, `logs --previous`, затем меняем **desired state** в Git/Helm и проверяем rollout.

26. Диагностика: симптом → причина → первая безопасная команда

Симптом	Что чаще всего означает	Первые команды	Чего не делать первым
`Pending`	Scheduler не нашёл подходящую node: request, selector, taint, PVC	`kubectl describe pod <pod> -n queue-system`	Не перезапускать pod: он ещё не запускался.
`ImagePullBackOff`	Image/tag недоступен node, registry auth/network	`kubectl describe pod <pod>` ; сверить rendered image	Не менять tag на `latest` наугад.
`CrashLoopBackOff`	Процесс завершается: ENV, command, panic, port, migration	`kubectl logs <pod> --previous` ; `describe pod`	Не запускать бесконечный rollout restart.
`OOMKilled`	Container превысил memory limit	`kubectl describe pod` ; metrics; profile memory	Не увеличивать limit без измерений/ причины.
`0/2 ready`	Probe/зависимость/запуск не прошли	`kubectl get pods` ; `kubectl describe deploy backend`	Не считать `Running` синонимом `Ready` .
Service 0 endpoints	Labels selector не совпал либо pod не ready	`kubectl get svc,endpointslice -n queue-system` ; проверить labels	Не хардкодить Pod IP.
502/504 от Ingress	Traefik не дошёл до Service/endpoints либо timeout	`kubectl describe ingress` ; endpoints; Traefik logs	Не сразу менять frontend.
Argo `OutOfSync`	Live отличается от Git desired state	Argo Diff; `argocd app diff queue-system`	Не делать ручной edit без понимания drift.
Argo `Degraded`	Один ресурс unhealthy, не вся система обязательно мертва	Открыть resource в Tree → `describe` /logs	Не нажимать Sync многократно.

Базовый investigation loop:

```
NS=queue-system
kubectl get pods,deploy,rs,svc,ingress -n "$NS" -o wide
kubectl get events -n "$NS" --sort-by='.metadata.creationTimestamp' | tail -n 30
kubectl describe pod -n "$NS" <pod-name>
kubectl logs -n "$NS" <pod-name> --all-containers --tail=150
kubectl logs -n "$NS" <pod-name> --previous --tail=150  # если restart был
```

`Events` — часто самый быстрый источник причины: `FailedScheduling`, `FailedMount`, `ErrImagePull`, `Unhealthy`. Они не хранятся вечно, поэтому важные инциденты фиксируют в тикете/логах.

27. GitOps, CI и release: кто за что отвечает

27.1. Чёткое разделение ролей

Стадия	Обычный ответственный инструмент	Результат
Проверка кода	CI (в проекте пока локально `go test`)	test report.
Build	Docker/BuildKit в CI	immutable image.
Хранение образа	Container registry	image доступен каждой node.
Изменение желаемой версии	Git PR/commit в values	audited Git revision.
Render/Sync	Argo CD + Helm	Kubernetes objects.
Запуск	K8s scheduler + kubelet + containerd	running Pod.
Контроль после deploy	smoke test, Prometheus/Grafana/alerts	evidence that release works.

Argo CD **не является CI**: он не обязан компилировать Go, запускать unit tests или строить image. Docker/registry **не являются CD**: они дают артефакт, но не говорят кластеру переключиться на него. Это популярная ловушка в вопросах.

27.2. Хорошая транзакция release

```
1. Developer merges application code
2. CI tests code and builds queue-backend:<git-sha>
3. CI scans and pushes image to registry
4. Bot/engineer changes only image digest/tag in GitOps repository
5. Review confirms rendered manifest/diff
6. Argo syncs desired Git revision
7. Kubernetes rolls out new RS while old remains serving
8. Smoke test + dashboards/alerts validate release
9. Failure → revert GitOps commit → Argo rolls back desired state
```

Отдельный GitOps repo необязателен для учебного проекта, но часто удобен: права на app code и право выкатывать production тогда можно разделить. В этом репозитории chart лежит рядом с кодом; это нормально, если protected branch и review discipline соблюдаются.

27.3. Rollout, rollback и четыре разных «версии»

Не смешивайте:

1. **Git commit** — история исходников/values.
2. **Docker image tag/digest** — какой бинарник запущен.
3. **Helm release revision** — история Helm installation (если Helm управляет release напрямую).
4. **Deployment/ReplicaSet revision** — история изменения Pod template.

Лучший GitOps rollback — revert commit с image tag и дождаться Argo Sync. Это сохраняет Git/live согласованными. `kubectl rollout undo deployment/backend` может быстро восстановить старый ReplicaSet, но затем надо зафиксировать

соответствующий desired state в Git, иначе Argo вернёт новую версию.

28. Security: базовая модель угроз для этой схемы

28.1. Что защищаем

credentials БД, внешних API, Telegram, OAuth, TLS private keys;
kubeconfig и service-account tokens — они дают доступ к кластеру;
container registry credentials;
персональные данные и backup PostgreSQL;
возможность изменить image tag/Helm values/Argo Application;
ingress и сетевые пути между services.

28.2. Минимальные правила

1. Секрет не хранится в Git, shell history, Dockerfile или Docker image layer.

2. При утечке token/password **ротируют**, а не только удаляют строку из файла.

3. Каждый человек/CI service account получает минимальные RBAC права.

4. Для production image закрепляют digest/tag, сканируют и берут из trusted registry.

5. TLS завершается в ingress; доступ к Grafana/Argo защищён SSO/RBAC, а не общим `admin` password.

6. NetworkPolicy ограничивает, кому доступны Postgres и observability ports.

7. Backup шифруют, ограничивают к нему доступ и регулярно делают restore test.

28.3. Почему Kubernetes Secret недостаточен сам по себе

`Secret` удобен, потому что Pod может прочитать его по ссылке и RBAC способен ограничить API access. Но `stringData` становится base64 `data`, а base64 не

шифрует. Security зависит от того, кто читает Git/etcd/API, включено ли encryption at rest и откуда secret попал в cluster. В текущем проекте пароли и tokens уже в tracked manifests, поэтому учебно важное улучшение — их ротация и внешний secret manager.

29. Scaling и отказоустойчивость — что реально означает «2 реплики»

Две backend replicas помогают, если одновременно верны условия:

```
оба pod готовы
AND Service имеет оба endpoint
AND Ingress/cluster network работают
AND PostgreSQL/внешние зависимости доступны
AND replicas не лежат на единственной упавшей node
```

В текущем values backend имеет 2 replicas, но `nodeSelector` целит их в control-plane. Если это одна node, две replicas защищают от падения процесса или rollout, но не от падения самой node. Для node-level HA нужны как минимум несколько eligible nodes, anti-affinity/topology spread и доступное shared/replicated storage. Это хороший зрелый ответ вместо обещания «у нас HA, потому что replicas: 2».

HPA, PDB и autoscaling простыми словами

HPA меняет `spec.replicas` по CPU/memory/custom metrics. Нужны metrics API, корректные requests и stateless API; HPA не решает capacity, если node уже полны.

Cluster autoscaler добавляет node, если cloud/infra позволяет; он не

заменяет HPA.

PDB задаёт, сколько pod приложения может быть добровольно недоступно при drain/maintenance. Он не спасает от внезапного падения node.

Anti-affinity/topology spread стараются разнести replicas по node/zone.

Их пока нет в chart — это нормальное обозначение следующего этапа, а не то, что стоит симулировать без метрик и capacity plan.

30. Практический чек-лист перед защитой

За день до защиты

[ ] Не показывать реальные токены, kubeconfig, IP/пароли; подготовить redacted screenshots/values.

[ ] Открыть Argo CD Tree и убедиться, что `Healthy` / `Synced` актуальны.

[ ] Выполнить `kubectl get deploy,rs,pods,svc -n queue-system` и понять, какой ReplicaSet текущий и почему старые имеют 0 pods.

[ ] Проверить `/health`, `/ready`, `/metrics`, один Grafana dashboard и один Jaeger trace (если компоненты реально запущены).

[ ] Подготовить вывод `helm template` или два файла рядом: values + template.

[ ] Знать различие current implementation vs roadmap: отдельные raw observability manifests, secrets в Git, отсутствующий registry/CI pipeline.

[ ] Не запускать destructive backup restore, `docker compose down`, prune или rollout во время демонстрации без заранее проверенного плана.

Во время демонстрации

```
Сначала — архитектура и задача.
Затем — один конкретный путь: values → Helm template → Argo → Deployment/RS/Pod.
Потом — один пользовательский запрос: Ingress → Service → backend → DB.
В конце — observability и 2–3 честных улучшения.
```

Если что-то live не открылось, не импровизируйте ложное объяснение. Скажите:

«Сейчас проверю desired/live через Tree и `kubectl describe`; это именно

стандартный incident workflow». Такой ответ демонстрирует инженерный подход.

31. Карманная шпаргалка команд

```
# Где я и какой кластер использую?
kubectl config current-context
kubectl config view --minify
kubectl get nodes -o wide

# Что живо в namespace?
kubectl get all -n queue-system
kubectl get pvc,ingress -n queue-system

# Почему конкретный pod не готов?
kubectl describe pod -n queue-system <pod>
kubectl logs -n queue-system <pod> --tail=100
kubectl logs -n queue-system <pod> --previous --tail=100

# Связь Service → endpoints → labels
kubectl get svc backend -n queue-system -o yaml
kubectl get endpointslice -n queue-system \
  -l kubernetes.io/service-name=backend -o yaml
kubectl get pods -n queue-system -l app=backend --show-labels

# Роллаут
kubectl rollout status deployment/backend -n queue-system
kubectl rollout history deployment/backend -n queue-system

# До применения посмотреть Helm-результат
helm lint helm/queue-chart
helm template queue-release helm/queue-chart > /tmp/queue-rendered.yaml

# Argo CD, если CLI установлен и авторизован
argocd app get queue-system
argocd app diff queue-system
argocd app history queue-system
```

Подставляйте реальное имя pod только после `kubectl get pods`; не копируйте старые имена ReplicaSet/Pod из скриншота — они намеренно изменяются при каждом rollout.